

Discussion Unit: DSE Analytics

Streaming, Streaming
Analytics

- Streaming use cases
- Blocking operators
- Streaming sources
- Reliable receivers
- Spark 2.0; production ready, RDD
- Spark 2.2; DataFrames, Datasets, windows, other
- Stateful, stateless transforms
- Window functions
- Window sizing
- Examples
- Writing

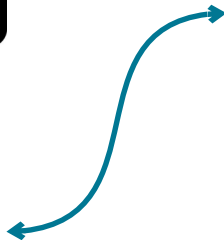


Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

Attributes of Streaming, Streaming Analytics

DSE Analytics: Streaming, Not-streaming



stdout/stderr (logs) stream from 100,000 servers, analyzed to correlate dependencies; will server-A issue impact server-B ?



Customer on the 800 number routed to specific customer service agent based on known conditions-



1200+ sensors on airplane, 4TB of data from 4 hour flight, monitored to predict failure before it happens



Inventory nearly gone, raise price before next price quote



Flow rate sensors on stream, it's raining hard; will it flood ?

SQL SELECT Clauses-

SELECT

o.order_num,
sum (i.total_price) price

FROM orders o, items i

JOINING

o.order_num = i.order_num

WHERE o.order_date > '01/01/89'

AND o.customer_num > 110

GROUP BY 1

ORDER BY 2

INTO TEMP my_tempTable;



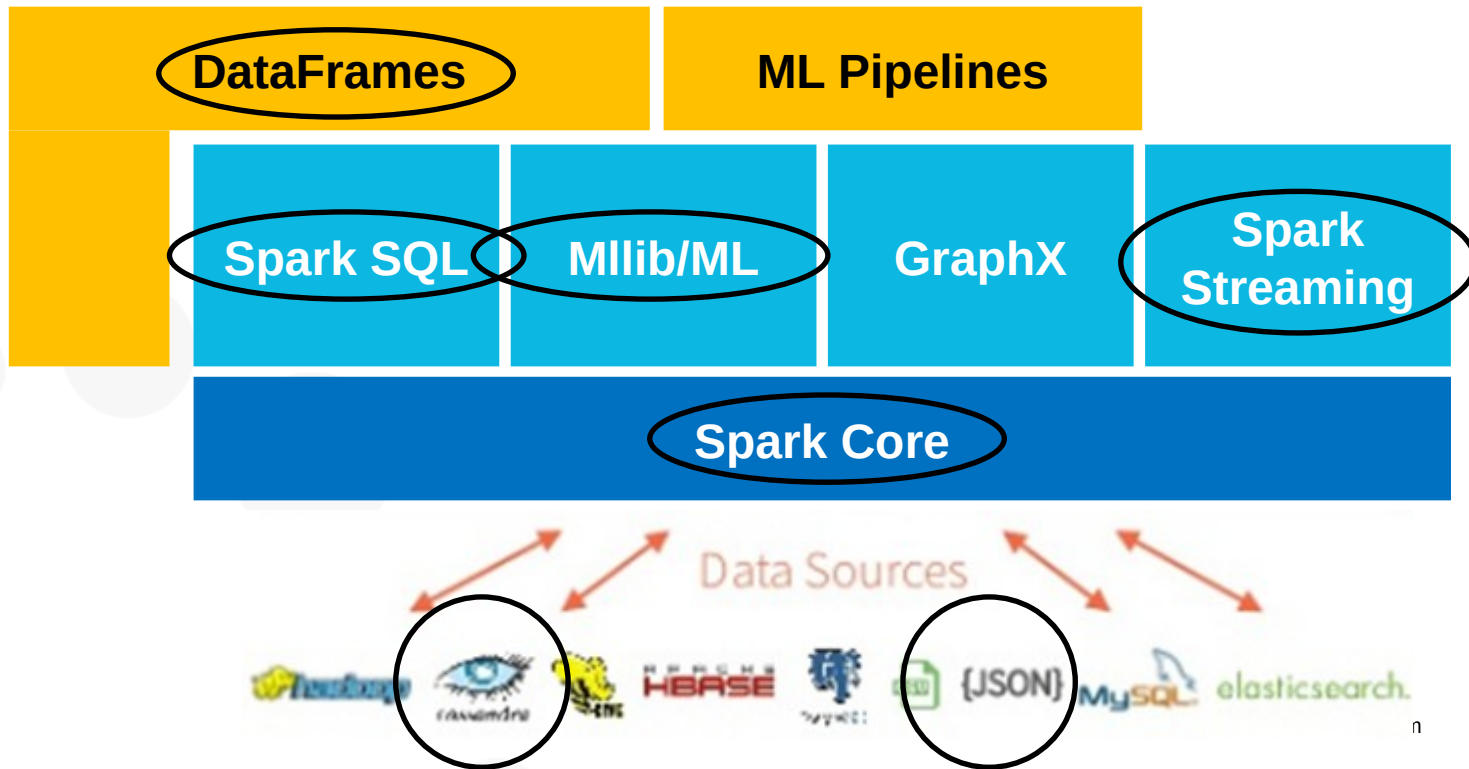
Describe the clauses ?
Which are blocking ?

(What is blocking ?)



End of Discussion Lab:

DSE Analytics: 5 Major Functional Areas



DSE Analytics/Streaming: Where to Use

Streaming is often utilized in a DSE deployment

- Monitor and act upon data as it is sent to DSE
- Always on query, versus polling

DSE Analytics Streaming sources:

- Kafka, ZeroMQ, MQTT
- JMS systems like MessageQ, ActiveMQ
- Twitter feeds
- Stock feeds
- CDC from other systems
- HDFS, DSEFS, S3
- Flume
- TCP sockets
- (Other)



Receiver Reliability: ACK/no-ACK, data loss is possible

DSE Analytics/Streaming: History

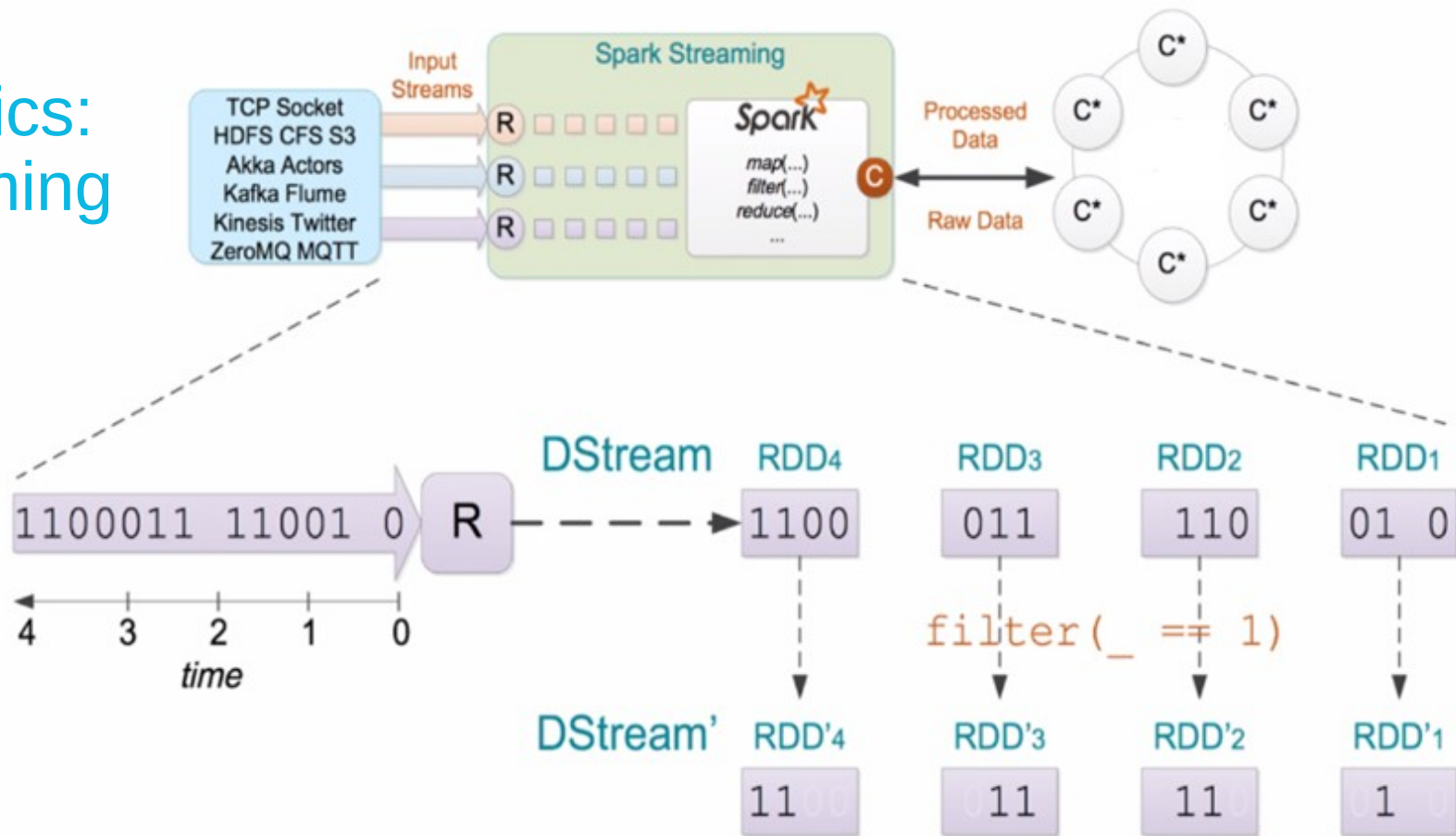
Spark 2.0 (DSE 5.1), Streaming 'production ready'

- RDD based, Dstreams (discretized)
- Micro batches, each being an RDD
- Fewer transforms (yes join, union; no intersect, ..)
- Highest number of articles, examples

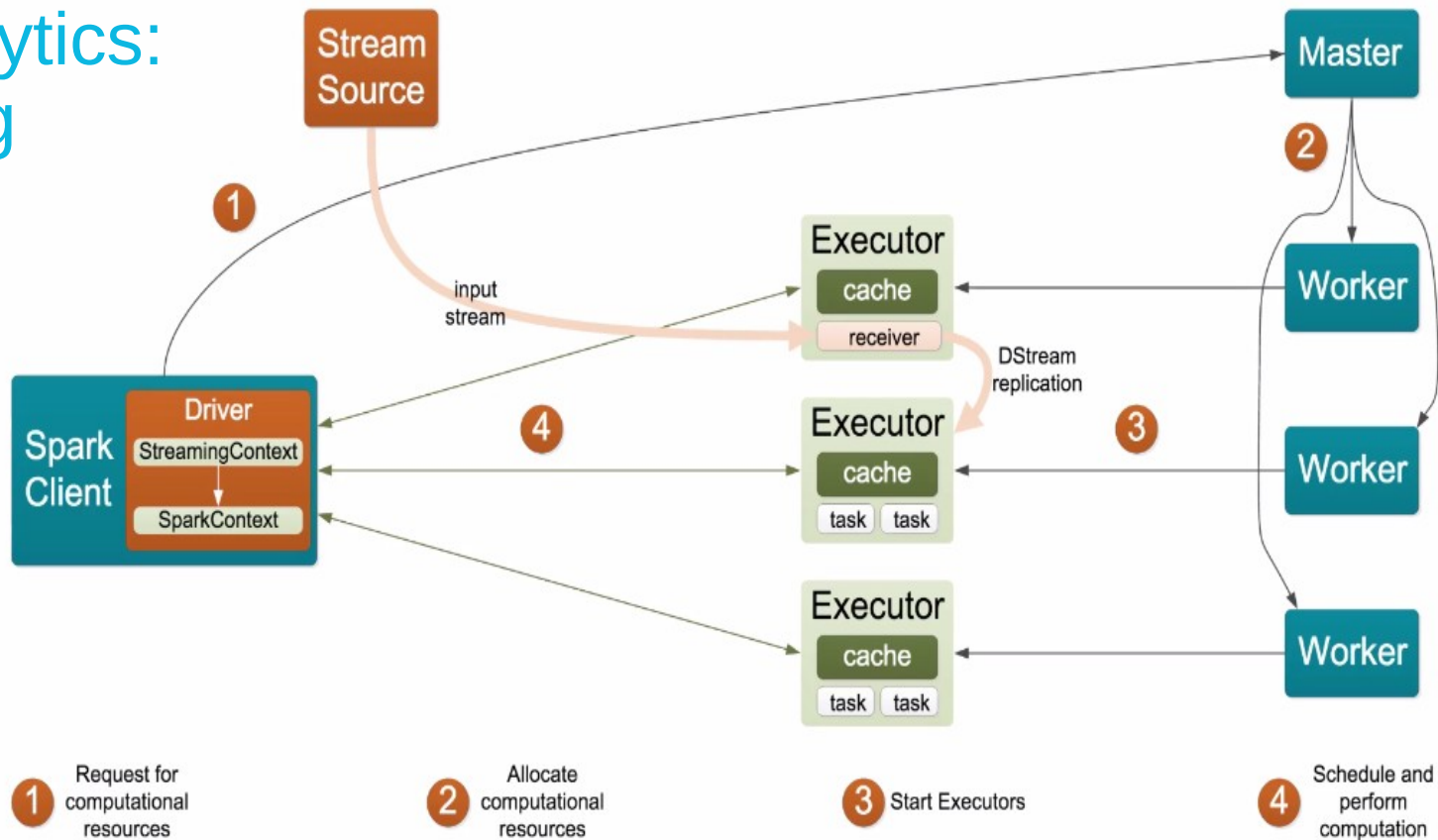
Spark 2.2 (DSE 6.0)

- DataFrame, Dataset
- Down to 1ms batch size
- Catalyst optimizer
- Automatic checkpointing
- State management automatic
- SQL access

DSE Analytics: Streaming

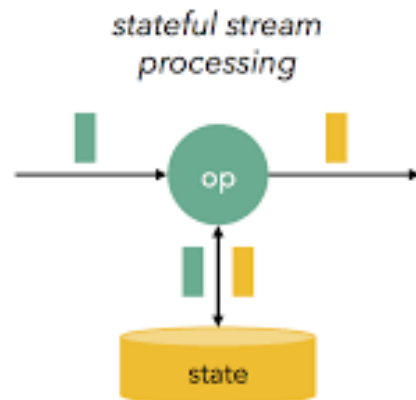


DSE Analytics: Streaming



DSE Analytics, Streaming

Stateless Transforms



Spark 2+, RDDs: DStream, Unary, Transform

Transformation	Description
filter(f)	New DStream, apply f() to elements of source Dstream, return where true
map(f)	New DStream, apply f() to each element, 1:1 input/output
flatMap(f)	New DStream, apply f(), 1:M input/output
count()	New DStream, single element, number of elements in source DStream
countByValue(n)	New DStream, count each distinct value in source, n == numTasks

Spark 2+, RDDs: DStream, Unary, Transform

Transformation	Description
<code>reduce(f)</code>	New DStream, produce single element which is aggregate from source DStream
<code>union(oDS)</code>	New DStream, similar to <code>union(oRDD)</code> , both DS must agree on time
<code>transform(f)</code>	New DStream, similar to <code>foreach()</code> , apply <code>f()</code> to every RDD in source DStream
<code>transformWith(oDS, f)</code>	Binary version of above. Both DStreams must agree on time

Spark 2+, Pair RDDs: DStream, Unary, Transforms

Transformation	Description
<code>mapValues(f)</code>	New DStream, <code>f()</code> , each element of pair RDD, known to preserve key, 1:1
<code>flatMapValues(f)</code>	Similar to above, 1:M
<code>reduceByKey(f [,n])</code>	Aggregate function <code>f()</code> , <code>n == num tasks</code>
<code>groupByKey(n)</code>	Aggregate, no custom function, <code>n == num tasks</code>
<code>combineByKey(..)</code>	Complex; combiner and merge <code>f()</code> 's, also merge combiners, can be written with multiples of above

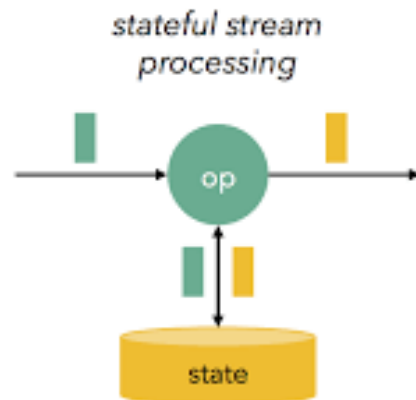
Spark 2+, Pair RDDs: DStream, Binary, Transforms

Transformation	Description
<code>cogroup(oDS [,n]))</code>	New DStream, similar to <code>groupByKey()</code>
<code>join(oDS [,n])</code>	New Dstream, similar to <code>join()</code>
<code>leftOuterJoin(..)</code>	Left outer
<code>rightOuterJoin(..)</code>	Right outer

** Each above: each participant must agree on time

DSE Analytics, Streaming

Stateful Transforms



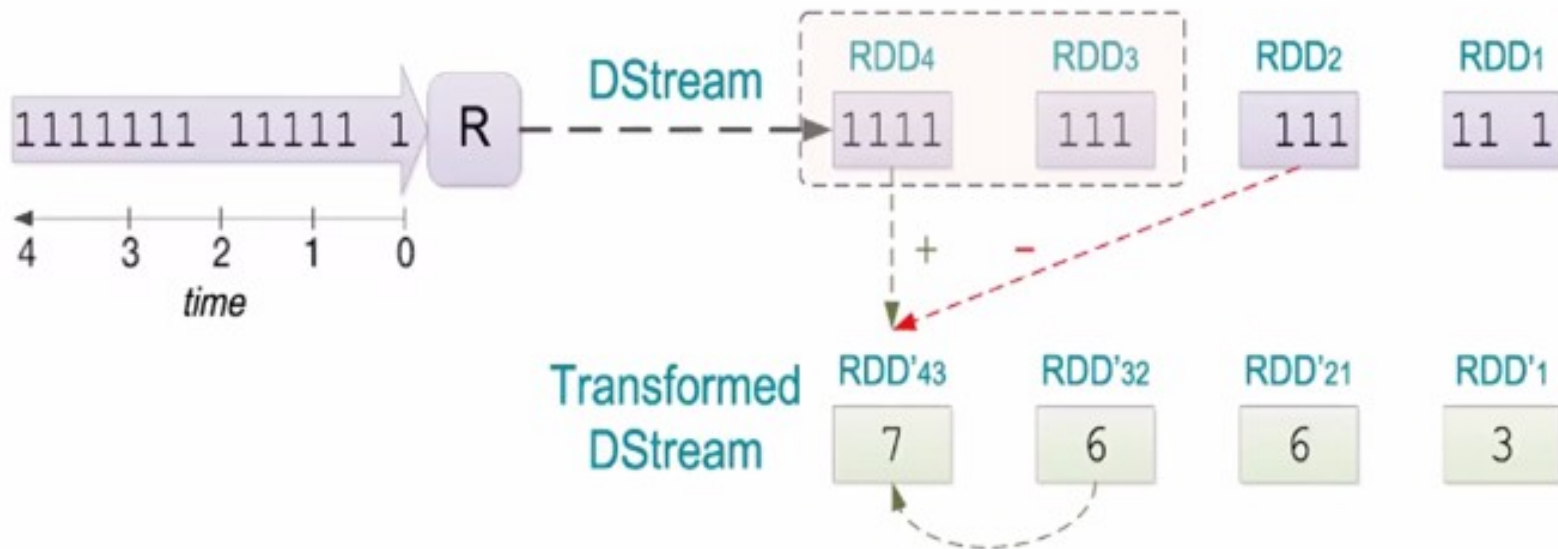
Spark 2+, RDDs: DStream, Unary, w/ State

Transformation	Description
<code>updateStateByKey(f [,n])</code>	New "state" DStream formed, apply f() on each key of source RDD, n == num tasks

** Above: must enable checkpointing

DSE Analytics: Windows, Slide or Tumble

`reduceByWindow(({a,b}  $\Rightarrow$  a+b), {(a,b}  $\Rightarrow$  a-b), 2 x batchInterval, batchInterval)`



Spark 2+, RDDs: DStream, Window Transforms

Transformation	Description
<code>window(l, [i])</code>	New DStream, combine all windowed batches from source DS, length, slideInterval
<code>countByWindow(l [,i])</code>	New DStream, Returns single element, count
<code>countByWindowAnd Value(l, i [,n])</code>	New DStream, Count for each distinct element in source DS
<code>reduceByWindow(f, l ,i)</code>	New DStream, apply f() to each element in source DS, aggregate
<code>reduceByWindow(..)</code>	Similar to above, second function to subtract, ie., running average

Spark 2+, Pair RDDs: DStream, Window Transforms

Transformation	Description
<code>reduceByKeyAndWindow(..)</code>	Similar to <code>reduceByKey</code> , Pair RDD, additive
<code>reduceByKeyAndWindow(..)</code>	Similar to above, add and subtract
<code>groupByKeyAndWindow(..)</code>	Aggregate, no function

DSE Analytics: Window Length

Window Length

- Minute scale window length reasonable
- Hour scale not recommended
- (Totals/Aggregates) okay

Window Sliding Interval

- Per application requirements



DSE Analytics, Examples



now now now is the time for all good persons
keyspaces can span what?

```
-----  
Time: 1533397030000 ms  
-----
```

```
(persons,1)  
(is,1)  
(,3)  
(now,3)  
(all,1)  
(good,1)  
(for,1)  
(time,1)  
(the,1)
```

```
-----  
Time: 1533397040000 ms  
-----
```

```
(can,1)  
(span,1)  
(keyspaces,1)  
(what?,1)
```

Example 1: Stateless, Word Count

- Sample input
- Sample output
- Second terminal,
nc -lk 9999

Example 1: Stateless, Word Count

```
package com.datastax.enablement.bootcamp
```

```
// Spark Core
```

```
import org.apache.spark.{SparkConf, SparkContext}
```

```
import org.apache.spark.SparkContext._
```

```
// Spark streaming
```

```
import org.apache.spark.streaming._
```

```
import org.apache.spark.streaming.StreamingContext._
```

```
// DSE Spark Cassandra Connector
```

```
import com.datastax.spark.connector._
```

```
import com.datastax.spark.connector.streaming._
```

Example 1: Stateless, Word Count

```
object App {  
  def main(args: Array[String]) {  
  
    val conf = new SparkConf(true).  
      setAppName("My App").  
      setMaster("dse://127.0.0.1?").  
      set("spark.cleaner.ttl", "3600")  
  
    val ssc = new StreamingContext(conf, Seconds(10))  
  
    val stream = ssc.socketTextStream("127.0.0.1", 9999)
```

Example 1: Stateless, Word Count

```
stream.flatMap(r => r.split(" ")).  
    map(t => (t, 1)).  
    reduceByKey(_ + _).  
    print()  
  
ssc.start()  
ssc.awaitTermination()  
  
}
```





IOT, Apache Http Web server

Example 2: Stateless, reduceByCount()

```
10.14.246.11 122.93.102.53 - - [25/Oct/2011:01:41:00 -0500]  
"GET /download/download6.zip HTTP/1.1" 200 3067  
"- " "Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; \  
rv:1.9.0.19) Gecko/2010031422 Firefox/3.0.19"
```

- Sample input

Example 2: Stateless, reduceByCount()

Time: 1533397460000 ms

(10.14.246.11, 2)
(10.14.246.12, 7)
(10.14.246.10, 1)

Time: 1533397470000 ms

(10.14.246.11, 3)
(10.14.246.13, 3)
(10.14.246.12, 4)

Time: 1533397480000 ms

(10.14.246.11, 2)
(10.14.246.12, 8)

- Sample output

Example 2: Stateless, reduceByCount()

```
#!/usr/bin/bash

l_cntr1=0
l_cntr2=0

function slow_cat() {
  while read
  do
    [ "$l_cntr1" -ge 10 ] && {
      sleep 10
      (>&2 echo " Loop Counter (10 rows output): "${l_cntr2})
      l_cntr1=0
    }
    #
    ((++l_cntr1))
    ((++l_cntr2))
    #
    echo "$REPLY"
  done
}
```

```
cat 02_WebLogAll.txt | slow_cat | nc -lk 9999
```



- Signal generator

Example 2: Stateless, reduceByCount()

```
package com.datastax.enablement.bootcamp
```

```
// Spark Core
```

```
import org.apache.spark.{SparkConf, SparkContext}
```

```
import org.apache.spark.SparkContext._
```

```
// Spark Streaming
```

```
import org.apache.spark.streaming._
```

```
import org.apache.spark.streaming.StreamingContext._
```

```
// DSE Spark Cassandra Connector
```

```
import com.datastax.spark.connector._
```

```
import com.datastax.spark.connector.streaming._
```

Example 2: Stateless, reduceByCount()

```
object App {  
  def main(args: Array[String]) {  
  
    // This area has changed alot since 5.x.  
    //  
    // Code below tested and works on 6.0  
    val conf = new SparkConf(true).  
      setAppName("My App").  
      setMaster("dse://127.0.0.1?").  
      set("spark.cleaner.ttl", "3600")  
  
    val ssc = new StreamingContext(conf, Seconds(10))  
  
    val stream = ssc.socketTextStream("127.0.0.1", 9999)
```

Example 2: Stateless, reduceByCount()

```
stream.map(r => (r.split(" ")(0), 1)).  
    reduceByKey(_+_).  
    print()
```

```
// Or, more simply
```

```
//
```

```
// stream.map(r => r.split(" ")(0)).
```

```
//     countByValue().
```

```
//     print()
```

```
ssc.start()
```

```
ssc.awaitTermination()
```

```
}
```

```
}
```



Time: 1533397610000 ms

(10.14.246.13, 10)
(10.14.246.10, 12)
(10.14.246.11, 1)
(10.14.246.12, 37)

Time: 1533397620000 ms

(10.14.246.13, 10)
(10.14.246.10, 13)
(10.14.246.11, 10)
(10.14.246.12, 37)

Example 3: Stateful, updateStateByKey()

- Sample output

```
package com.datastax.enablement.bootcamp
```

Example 3: Stateful, updateStateByKey()

```
// Spark Core
```

```
import org.apache.spark.{SparkConf, SparkContext}
```

```
import org.apache.spark.SparkContext._
```

```
// Spark Streaming
```

```
import org.apache.spark.streaming._
```

```
import org.apache.spark.streaming.StreamingContext._
```

```
// DSE Spark Cassandra Connector
```

```
import com.datastax.spark.connector._
```

```
import com.datastax.spark.connector.streaming._
```

Example 3: Stateful, updateStateByKey()

```
object App {  
  
  def main(args: Array[String]) {  
  
    val conf = new SparkConf(true).  
      setAppName("My App").  
      setMaster("dse://127.0.0.1?").  
      set("spark.cleaner.ttl", "3600")  
  
    val ssc = new StreamingContext(conf, Seconds(10))  
    //  
    ssc.checkpoint("dsefs:///tmp")  
  
    val stream = ssc.socketTextStream("127.0.0.1", 9999)
```



Example 3: Stateful, updateStateByKey()

```
def updateRunningCount(newValues: Seq[Long],  
    oldValues: Option[Long]) : Option[Long] = {  
    if (newValues.isEmpty)  
        Some(oldValues.getOrElse(0L))  
    else  
        Some(oldValues.getOrElse(0L) + newValues(0))  
}
```

```
stream.map(r => r.split(" ")(0)).  
    countByValue().  
    updateStateByKey[Long](updateRunningCount _).  
    print()
```



Example 3: Stateful, updateStateByKey()

```
// stream.map(r => r.split(" ")(0)).
//     reduceByWindow({ (nv, ov) => nv+ov}, Seconds(60),
//         Seconds(10)).
//     print()
//
// Essentially a concat of the IP addr; not useful

ssc.start()
ssc.awaitTermination()

}

}
```

Example 4: Stateful, with initial values

Time: 1533398970000 ms

(10.14.246.13, 1087)
(10.14.246.13, 1088)
(10.14.246.13, 1089)
(10.14.246.11, 86)
(10.14.246.12, 633)
(10.14.246.12, 634)
(10.14.246.12, 635)
(10.14.246.12, 636)
(10.14.246.12, 637)
(10.14.246.12, 638)

- Sample output

Example 4: Stateful, with initial values

```
package com.datastax.enablement.bootcamp

// Spark Core
import org.apache.spark.{SparkConf, SparkContext}
import org.apache.spark.SparkContext._

// Spark Streaming
import org.apache.spark.streaming._
import org.apache.spark.streaming.StreamingContext._

// DSE Spark Cassandra Connector
import com.datastax.spark.connector._
import com.datastax.spark.connector.streaming._
```

Example 4: Stateful, with initial values

```
object App {  
  
  def main(args: Array[String]) {  
  
    val conf = new SparkConf(true).  
      setName("My App").  
      setMaster("dse://127.0.0.1?").  
      set("spark.cleaner.ttl", "3600")  
  
    val ssc = new StreamingContext(conf, Seconds(10))  
      //  
    ssc.checkpoint("dsefs:///tmp")  
  
    val lines = ssc.socketTextStream("127.0.0.1", 9999)
```

Example 4: Stateful, with initial values

```
val initialRDD = ssc.sparkContext.parallelize(List(
    ("10.14.246.12", 500), ("10.14.246.13", 1000)))

val wordDstream = lines.map(r => (r.split(" ")(0), 1))

val mappingFunc = (word: String, one: Option[Int],
    state: State[Int]) => {

    val sum = one.getOrElse(0) + state.getOption.getOrElse(0)
    val output = (word, sum)
    state.update(sum)
    output
}
```

Example 4: Stateful, with initial values

```
val stateDstream = wordDstream.mapWithState(  
    StateSpec.function(mappingFunc).initialState(initialRDD))  
  
stateDstream.print()  
  
ssc.start()  
ssc.awaitTermination()  
  
}  
  
}
```

Example 5: Datasets (stateful) G/A July 2, 2018

```
// -----  
// Batch: 29  
// -----  
// +-----+-----+  
// |          value|count|  
// +-----+-----+  
// |10.14.246.11|   69|  
// |10.14.246.12|  177|  
// |10.14.246.13|   63|  
// |10.14.246.10|   71|  
// +-----+-----+  
//  
// -----  
// Batch: 30  
// -----  
// +-----+-----+  
// |          value|count|  
// +-----+-----+  
// |10.14.246.11|   78|  
// |10.14.246.12|  177|  
// |10.14.246.13|   63|  
// |10.14.246.10|   72|  
// +-----+-----+
```

- Sample output

```
package com.datastax.enablement.bootcamp

import org.apache.spark.sql.Session

object App {

  def main(args: Array[String]) {

    val host = "localhost"
    val port = 9999

    val spark = Session
      .builder
      .appName("My App")
      .getOrCreate()

    import spark.implicits._
```

Example 5: Datasets (stateful)

Example 5: Datasets (stateful)

```
val lines = spark.readStream
    .format("socket")
    .option("host", host)
    .option("port", port)
    .load()

val words = lines.as[String].map(r => r.split(" ")(0))

// println(words.getClass)
// class org.apache.spark.sql.Dataset

val wordCounts = words.groupBy("value").count()

val query = wordCounts.writeStream
    .outputMode("complete")
    .format("console")
    .start()
```

Example 5: Datasets (stateful)

```
query.awaitTermination()  
}
```

```
}
```

DSE Analytics: Spark 2.2+

- For all new work
- G/A July 2, 2018
- DataFrames/Datasets
- Auto checkpoint/stateful
- More complete transforms

Structured Streaming Programming Guide

- Overview
- Quick Example
- Programming Model
 - Basic Concepts
 - Handling Event-time and Late Data
 - Fault Tolerance Semantics
- API using Datasets and DataFrames
 - Creating streaming DataFrames and streaming Datasets
 - Input Sources
 - Schema inference and partition of streaming DataFrames/Datasets
 - Operations on streaming DataFrames/Datasets
 - Basic Operations - Selection, Projection, Aggregation
 - Window Operations on Event Time
 - Handling Late Data and Watermarking
 - Join Operations
 - Streaming Deduplication
 - Arbitrary Stateful Operations
 - Unsupported Operations
 - Starting Streaming Queries
 - Output Modes
 - Output Sinks
 - Using Foreach
 - Managing Streaming Queries
 - Monitoring Streaming Queries
 - Interactive APIs
 - Asynchronous API
 - Recovering from Failures with Checkpointing
- Where to go from here

DSE Analytics: Other

Parallelism

- Number of receivers invoked
- Number of partitions (same as before)

Persistence

- Same as before (cache(), ..)

Checkpointing

- Some transforms auto-checkpoint
- `updateStateByKey( .. )`
- `countByWindow( .. )`
- (Other)
- All window transforms

```
ssc.checkpoint("dsefs:///tmp")
```

```
ssc.checkpoint(Seconds(20))
```

- Frequent, slower perf
- Infrequent, slower failure recovery
- Recommend: 5-10 x slide interval

DSE Analytics: Writing

Output Operation	Description
<code>print()</code>	Print first 10 elements of each RDD in the source DStream
<code>saveAsTextFiles()</code> <code>saveAsObjectFiles()</code> <code>saveAsHadoopFiles()</code>	Save each DStream in format/location specified.
<code>foreachRDD(f)</code>	Apply <code>f()</code> to each RDD in source DStream. Can use any transform/actions on input RDD.
<code>saveToCassandra()</code>	Write to DSE Core; <code>ks</code> , <code>table</code> , <code>[columns]</code>



End of Unit: